

Trie | (Delete)

In the [previous post](#) on [trie](#) we have described how to insert and search a node in trie. Here is an algorithm how to delete a node from trie.

During delete operation we delete the key in bottom up manner using recursion. The following are possible conditions when deleting key from trie,

1. Key may not be there in trie. Delete operation should not modify trie.
2. Key present as unique key (no part of key contains another key (prefix), nor the key itself is prefix of another key in trie). Delete all the nodes.
3. Key is prefix key of another long key in trie. Unmark the leaf node.
4. Key present in trie, having atleast one other key as prefix key. Delete nodes from end of key until first leaf node of longest prefix key.

The below code presents algorithm to implement above conditions.

```
// C++ implementation of delete operations on Trie
#include <bits/stdc++.h>
using namespace std;

const int ALPHABET_SIZE = 26;

// trie node
struct TrieNode {
    struct TrieNode* children[ALPHABET_SIZE];

    // isEndOfWord is true if the node represents
    // end of a word
    bool isEndOfWord;
};

// Returns new trie node (initialized to NULLs)
struct TrieNode* getNode(void)
{
    struct TrieNode* pNode = new TrieNode;

    pNode->isEndOfWord = false;

    for (int i = 0; i < ALPHABET_SIZE; i++)
        pNode->children[i] = NULL;

    return pNode;
}

// If not present, inserts key into trie
// If the key is prefix of trie node, just marks leaf node
void insert(struct TrieNode* root, string key)
```

```
{
    struct TrieNode* pCrawl = root;

    for (int i = 0; i < key.length(); i++) {
        int index = key[i] - 'a';
        if (!pCrawl->children[index])
            pCrawl->children[index] = getNode();

        pCrawl = pCrawl->children[index];
    }
    // mark last node as leaf
    pCrawl->isEndOfWord = true;
}

// Returns true if key presents in trie, else
// false
bool search(struct TrieNode* root, string key)
{
    struct TrieNode* pCrawl = root;

    for (int i = 0; i < key.length(); i++) {
        int index = key[i] - 'a';
        if (!pCrawl->children[index])
            return false;

        pCrawl = pCrawl->children[index];
    }
    return (pCrawl != NULL && pCrawl->isEndOfWord);
}

// Returns true if root has no children, else false
bool isEmpty(TrieNode* root)
{
    for (int i = 0; i < ALPHABET_SIZE; i++)
        if (root->children[i])
            return false;
    return true;
}

// Recursive function to delete a key from given Trie
TrieNode* remove(TrieNode* root, string key, int depth = 0)
{
    // If tree is empty
    if (!root)
        return NULL;

    // If last character of key is being processed
    if (depth == key.size()) {

        // This node is no more end of word after
        // removal of given key
```

```

        if (root->isEndOfWord)
            root->isEndOfWord = false;

        // If given is not prefix of any other word
        if (isEmpty(root)) {
            delete (root);
            root = NULL;
        }

        return root;
    }

    // If not last character, recur for the child obtained using ASCII value
    int index = key[depth] - 'a';
    root->children[index] =
        remove(root->children[index], key, depth + 1);

// If root does not have any child (its only child got deleted), and it is not end of another word.
    if (isEmpty(root) && root->isEndOfWord == false) {
        delete (root);
        root = NULL;
    }
    return root;
}

int main() // Driver
{
    // Input keys (use only 'a' through 'z'
    // and lower case)
    string keys[] = { "the", "a", "there",
                     "answer", "any", "by",
                     "bye", "their", "hero", "heroplane" };
    int n = sizeof(keys) / sizeof(keys[0]);

    struct TrieNode* root = getNode();

    // Construct trie
    for (int i = 0; i < n; i++)
        insert(root, keys[i]);

    // Search for different keys
    search(root, "the") ? cout << "Yes\n" : cout << "No\n";
    search(root, "these") ? cout << "Yes\n" : cout << "No\n";

    remove(root, "heroplane");
    search(root, "hero") ? cout << "Yes\n" : cout << "No\n";
    return 0;
}

```
